

Project Report: Design of a Single-cycle ARM Processor (Verilog)

Garrett Good, Joyce Berdkan

School of Engineering, Liberty University

ENG 361: Computer Architecture

Professor Feng Wang

December 02, 2024

Abstract

The central focus of this project was building a single-cycle ARM processor using Verilog with the help of five previous labs which served as the foundational building blocks. We were tasked with designing the rest of the units of the processor and the requirements of this project included being to execute the following instructions: ADD, SUB, AND, ORR, LDR, STR, and B. Furthermore, we were to create a table and list our predictions of what will happen at each cycle as well, so that we could ensure our ModelSim waveform outputs were as expected. After building the program, we were able to run it and compare the waveform to our table; it was a successful single-cycle ARM processor. In addition, a modified ARM processor was built as well with the hopes of running two additional instructions, EOR and LDRB. Once again we created a table to compare with our ModelSim waveform outputs, and the result seemed to be as desired.

Table of Contents

Introduction.....	3
Background.....	5
Specification.....	7
Design.....	9
Implementation.....	10
Results and Evaluation.....	11
Future Works.....	12
Conclusions.....	13
Reflection of Learning.....	14
References.....	15
Appendix.....	16

Introduction

This report details the steps in which we will take to design and build a single-cycle ARM processor using Verilog. Throughout the past few weeks, we have conducted 5 unique labs that will be used as building blocks to set up our final design. The end goal of this project will be a fully designed and implemented basic single-cycle ARM processor that can execute a set of given instructions.

The intended audience of this report is our professor, Dr. Wang; we aim to show him our ability to identify, formulate, and solve a complex engineering problem, such as this one which involves the building of a single-cycle ARM processor from the ground up. Problem identification begins with recognizing that designing a single-cycle ARM processor involves a variety of interrelated components, including relevance to complex engineering problems, modern engineering problems, and educational value. Designing an ARM processor is not only a complex issue, but also a modern one, as technology constantly advances and new ways of doing things continue to evolve exponentially, which is why the educational gain from this process is so fulfilling.

To formulate the problem, it is a must to create a structured framework for addressing such a complex task- defining the scope, objectives, constraints, and assumptions are all part of the process. The scope of the project includes the design, implementation, and testing of a basic single-cycle ARM processor that executes a set of specific instructions. The objectives of this project are to first design a single-cycle ARM process and implement various instructions, then ensure the design is functional using a predetermined testing program, and finally modify the processor to include two additional instructions and ensure its' functionality if desired. The technical constraints include all instructions must be executed in one clock cycle, bounded to a set of ARM instructions, and must be implemented using Verilog. The design constraints include using certain Datapath instructions, Memory instructions, and Branch instructions, and then creating a testbench to view results. The testing constraints include implementing a given test bench while altering the instruction memory file, and then modifying another given testing program by converting the program to machine language and putting it into the instruction memory.

One assumption that can be made about our single-cycle ARM processor design involves the CPU's clock speed- all instructions are to be completed within a single clock cycle. Furthermore, the instruction memory will have pre-defined machine code and all data memory operations in this study assume their to be no delays. The last assumption to be made involve the subset of ARM instructions the processor must support:

- Datapath (DP) instructions: ADD, SUB, AND, ORR
- Memory instructions: LRD, STR
- Branch instructions: B

Figure 1 shows the single-cycle ARM processor schematic of our design that is capable of executing each of these various instructions listed above. These various instructions will be investigated in much further detail later in the report; this just sets the table for what our resulting ARM processor will be capable of.

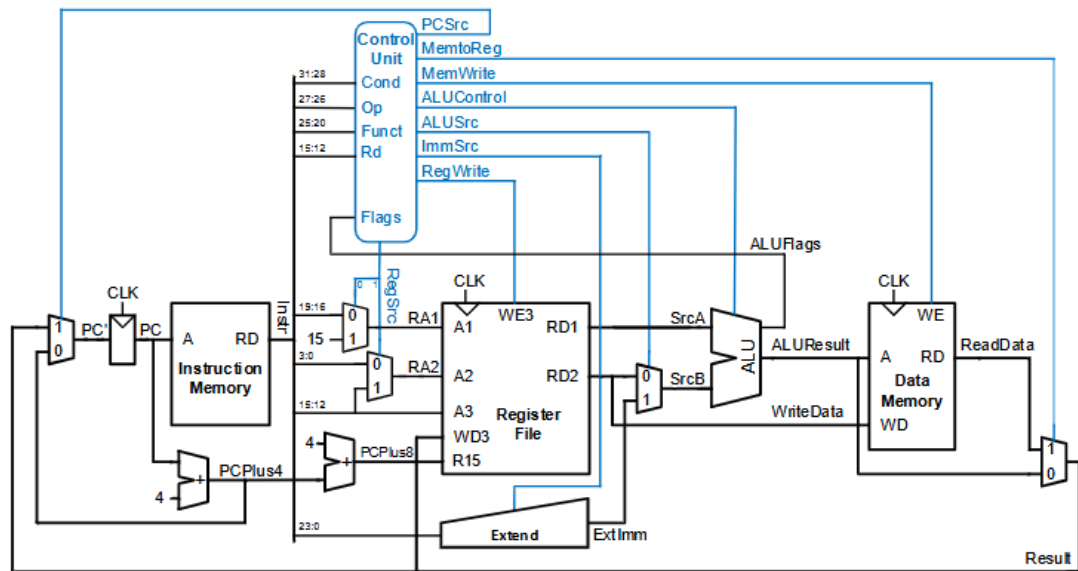


Figure 1. Single-cycle ARM Processor

Furthermore, due to this being a complex system, our team will describe our predictions of what will happen on each cycle when running the program in a table to verify the ModelSim results that will be gathered after running the program. This process of matching our predictions with the results from ModelSim will help ensure our ARM processor is running as intended. The end outcome that will be evaluated is determining if our designed single-cycle ARM processor meets the needs and requirements of all the aspects of this project.

Background

As stated earlier, this project will be using 5 previous labs as building blocks in the designing of this single-cycle ARM processor. These labs include creating a file for a 32-bit wide 2-to-1 multiplexer, a 32-bit Arithmetic Logic Unit (ALU), 3 files implementing the register file, instruction memory, and data memory that will be used in the processor, the creation of the Datapath and its' various files, and lastly a decoder file and its' components. These various files will be combined into a singular project file and will be the foundation for our single-cycle ARM processor.

A 2-to-1 multiplexer chooses between 2 data inputs based on the select, which is also called the control signal. In figure 2, "s" is the control signal, "x" and "y" are the inputs, and "m" is the output. In this diagram, if $s = 0$, then $x = m$ and if $s = 1$, then $y = m$. The purpose of a multiplexer is for switching one of several digital signals to an output, under the control of another set of binary inputs.

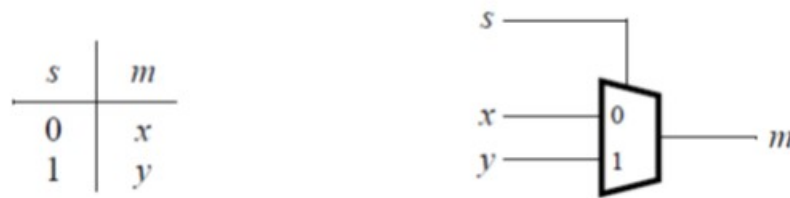
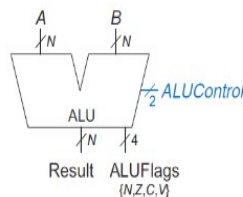


Figure 2. A 2-to-1 Multiplexer

An ALU merges various logical and mathematical operations into a single unit. Some of these operations include addition, subtraction, AND, and OR. Furthermore, the ALU also builds the ALUFlags output, which is composed of the negative (N), zero (Z), carryout (C), and overflow (V) flags. In figure 3, the diagram shows how the control function informs the ALU what execution result to output.



Inputs	
Port	Description
A(31:0)	Operand A is a 32 bit input bus carrying the "A" operand for an ALU operation.
B(31:0)	Operand B is a 32 bit input bus carrying the "B" operand for an ALU operation.
ALUControl (1:0)	Operation is a 2 bit input bus which controls which operation the ALU executes.

Outputs	
Port	Description
Result(31:0)	Result is a 32 bit output bus which will hold the result of the ALU Operation.
ALUFlags(3:0)	ALUFlags output is composed of the N, Z, C, and V flags that indicate, respectively

Figure 3. 32-Bit Arithmetic Logic Unit (ALU)

The register file implemented into this project holds 15 registers (R0-R14) and an additional input to receive another register (R15) from the program counter (PC). It also has two read ports, RD1 and RD2, and one write port, WE3. The instruction memory stores the instructions the processor will execute. The data memory file stores the data the processor accesses.

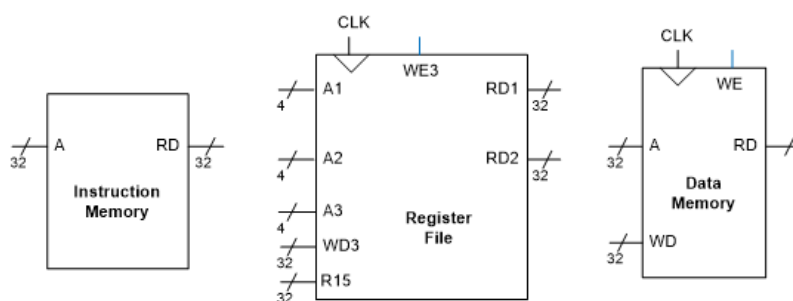


Figure 4. 32-Bit register file with 16 registers; Instruction and Data memory

The Datapath file used incorporated parts of the previous labs in order to build the datapath of a single-cycle ARM processor. In addition, it also built a program counter register and two 32-bit adders. Once all the components were built, we were able to instantiate all of the modules inside the datapath file.

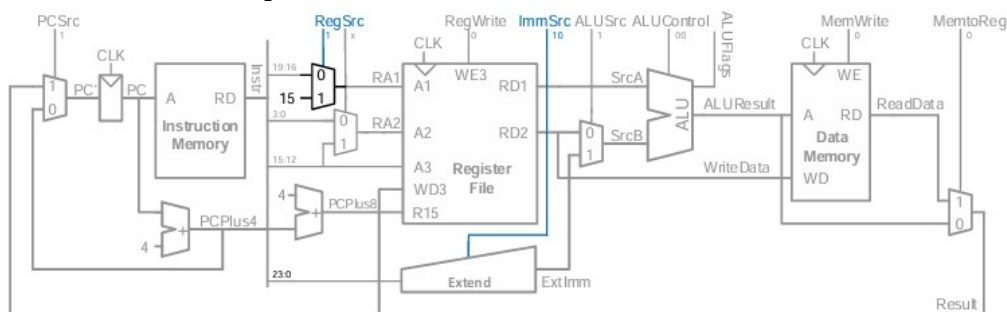
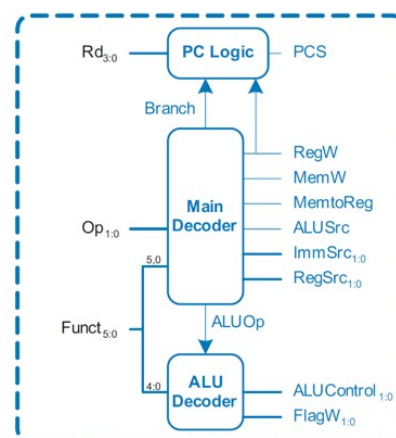


Figure 5. Datapath schematic

The final piece of our five building blocks was building the decoder of a single-cycle ARM processor, including the main decoder, ALU decoder, and PC logic. The main decoder produces most of the control signals, the ALU decoder determines the type of data-processing instruction, and the PC logic determines whether the PC needs updating. Figure 6 depicts the schematic of the decoder in its entirety.

Figure 6. Single-Cycle Control Unit: Decoder



Specification

1. Instructions

We are required to design and build an ARM single-cycle processor using Verilog. Essential data processing commands including ADD (addition), SUB (subtraction), AND (bitwise AND), ORR (bitwise OR), LDR (load register from memory), STR (store register to memory), and B (branch) should all be supported by the processor. EOR (bitwise XOR) and LDRB (load byte to register) must also be present in the additional credit version. The processor combines a control unit to coordinate all parts by producing the right signals according to the kind of instruction, a data memory module for read and write operations, and instruction memory to store the program. The assignment contains a detailed list of instructions that must be followed in order for the processor to function properly. The base assignment requires the following instructions to run:

```

MAIN      SUB  R0, R15, R15
          ADD  R2, R0, #5
          ADD  R3, R0, #12
          SUB  R7, R3, #9
          ORR  R4, R7, R2
          AND  R5, R3, R4
          ADD  R5, R5, R4
          SUBS R8, R5, R7
          BEQ  END
          SUBS R8, R3, R4
          BGE  AROUND
          ADD  R5, R0, #0
AROUND    SUBS R8, R7, R2
          ADDLT R7, R5, #1
          SUB  R7, R7, R2
          STR  R7, [R3, #84]
          LDR  R2, [R0, #96]
          ADD  R15, R15, R0
          ADD  R2, R0, #14
          B    END
          ADD  R2, R0, #13
          ADD  R2, R0, #10
END        STR  R2, [R0, #100]

```

For the extended portion of this assignment, both EOR and LDRB commands are added to the list of instructions

```

MAIN      SUB  R0, R15, R15
          ADD  R1, R0, #255
          ADD  R2, R1, R1
          STR  R2, [R0, #196]
          EOR  R3, R1, #77
          AND  R4, R3, #0x1F
          ADD  R5, R3, R4

```

```

        LDRB R6, [R5]
        LDRB R7, [R5, #1]
        SUBS R0, R6, R7
        BLT  MAIN
        BGT  HERE
        STR  R1, [R4, #110]
        B    MAIN
HERE      STR  R6, [R4, #110]

```

2. Modules

The ARM module, Data memory, and Instruction memory are the three primary top-level modules that make up the system. The system's controller and datapath are located in the ARM module. Words of data are used by the datapath. ALUs, extend modules, adders, multiplexors, floppers, and register files are among the structures it includes. We utilize a 32-bit datapath because we are implementing the 32-bit ARM architecture. The control unit instructs the datapath on how to carry out the current instruction after receiving it from the datapath. The cond, op, and funct fields of the instruction, along with the flags and whether the destination register is the PC, are used by the control unit to construct the control signals. Additionally, the controller keeps track of the status flags and makes the necessary modifications. The controller consists of a decoder unit that creates the control signals based on Instr and a conditional logic unit that maintains the status flags and only permits updates to the architectural state when the instruction to be executed.

There is only one read/write port on the data memory. It makes it possible for the processor to store information in registers and load information from memory into registers. Memory-related instructions like LDR, LDRB, and STR are executed smoothly thanks to its interaction with the ALU and control unit.

There is only one read port on the instruction memory. The instruction memory, register file, and data memory are all read combinational. It accepts a 32-bit instruction address, A, then reads the 32-bit instruction from that location onto the read data output, Rd. Put another way, if the address changes, there is no clock involved and the new data appears at Rd after a certain amount of propagation delay. Only the rising edge of the clock has them written. Only at the clock edge is the system's state altered in this way. Prior to the clock edge, the address, data, and write enable must be set up, and they must stay stable until a hold period following the clock edge.

Design

1. Constraints

The requirement of one clock cycle per instruction is the most important constraint in this system. This places restrictions on the amount of processing or data transmission that can take place in a single cycle, directing choices to give simplicity and efficiency top priority. To restrict the complexity of processes, the CPU, for example, uses combinational logic to compute ALU results and generate control signals in a single clock cycle. In order to prevent conflicts, resource sharing across phases must also be carefully organized.

2. Single-Cycle Datapath

The address of the instruction to be executed is stored in the program counter. Reading this instruction memory is the initial step. The PC is merely connected to the instruction memory's address input. The 32-bit instruction, designated Instr, is read out, or retrieved, from the instruction memory.

The execution of an LDR instruction with a positive immediate offset involves several key datapath connections and operations. Initially, the source register containing the base address is read from the register file using the Rn field Instr19:16, with its value sent to the RD1 port. The offset for the instruction, stored in the immediate field Instr11:0, is zero-extended to 32 bits, creating the ExtImm value. The ALU computes the effective memory address by adding the base address SrcA and the offset SrcB, with the operation specified by the ALUControl signal set to 00 for addition. The resulting address ALUResult is sent to the data memory, which outputs the data onto the ReadData bus. This data is written back to the destination register, specified by the Rd field Instr15:12, via the WD3 input of the register file. The RegWrite signal is asserted to enable writing.

During this process, the program counter PC is updated to point to the next instruction PC+4 using an adder. For special cases where the base or destination register is R15 (the PC), an additional adder increments the PC by 8 (PC+8), and a multiplexer selects between the new PC value PCPlus4 or ReadData, depending on the PCSrc control signal. this flexible datapath design, including zero-extension, address computation, and condition PC updates, ensures the correct execution of the LDR instruction and supports further generalizations for additional ARM instructions.

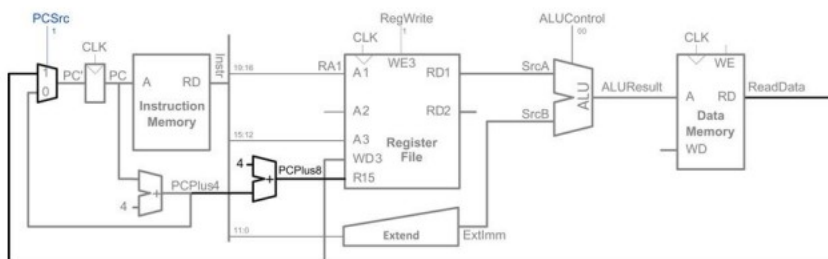


Figure 7. Read or write program counter as R15

To handle the differing outputs of LDR and data-processing instructions, a multiplexer is added to select between ReadData and ALUResult. This multiplexer determines the final value (Result) written to the register file. The new control signal MemtoReg governs this selection: for LDR, MemtoReg = 1 selects ReadData, and for data-processing instructions, MemtoReg=0 selects ALUResult. Since STR does not write to the register file, the value of MemtoReg is irrelevant for STR. Additionally, for data-processing instructions using register addressing, the second ALU source is taken from the Rm register specified by Instr3:0. To accommodate this, multiplexers are added to the register file and the ALU inputs to select the appropriate second operand. These updates enable the datapath to handle both immediate and register-based addressing modes effectively while maintaining shared hardware resources.

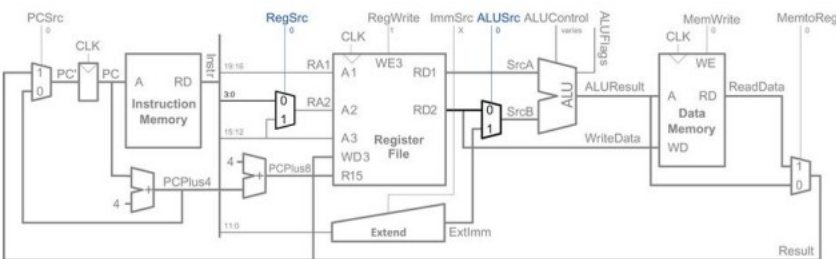


Figure 10. Datapath enhancements for data-processing instructions with register addressing

The datapath incorporates flexible operand selection based on control signals. For the STR instruction, the second register address (RA2) is derived from the Rd field (Instr15:12), while for data-processing instructions with register addressing, it comes from the Rm field (Instr3:0), as controlled by the RegSrc signal. Similarly, the second ALU source is chosen wither from the register file for register-addressing instructions or from zero-extended immediate (ExtImm) for immediate-addressing instructions, based on the ALUSrc control signal.

To support the B (branch) instruction, the datapath is further extended. The branch instruction computes the target address by adding a 24-bit immediate (multiplied by 4 and sign-extended) to PC+8. This requires an additional mode for the Extend block to handle sign extension for branch immediates. Two additional bits are added to the ImmSrc control signal to support this functionality, ensuring seamless handling of branch instructions in the expanded datapath.

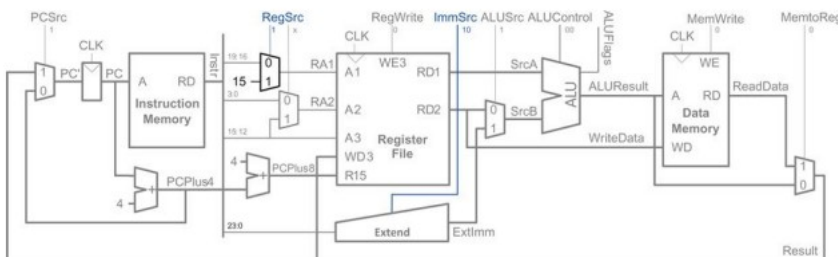


Figure 11. Datapath enhancements for B instruction

The register file's first port is used to read PC+8. Consequently, in order to select R15 as the RA1 input, a multiplexer is required. Another byte of RegSrc controls this multiplexer, selecting Instr19:16 for the majority of instructions but 15 for B. The new PC is chosen from ALUResult for the branch by setting PCSrc to 1 and MemtoReg to 0. This completes the datapath design for the single-cycle CPU.

Implementation

After clarifying the specifications and creating a base design, it was time to finally implement all of the components together. The ARM single-cycle processor was implemented as a modular system consisting of interconnected Verilog modules for the ALU, data memory, instruction memory, control unit, and datapath.

1. LDRB

Main Decoder truth table with LDRB implemented and RegW

Op	Func _{t₅}	Func _{t₀}	Type	Branch	MemtoReg	MemW	ALUSrc	ImmSrc	RegW _{1:0}	RegSrc	ALUOp
00	0	X	DP Reg	0	0	0	0	XX	1X	00	1
00	1	X	DP Imm	0	0	0	1	00	1X	X0	1
01	X	0	STR	0	X	1	1	01	0X	10	0
01	X	1	LDR	0	1	0	1	01	1X	X0	0
01	X	1	LDRB	1	1	0	1	11	11	X0	0
10	X	X	B	1	0	0	1	10	0X	X1	0

Table 1. Main Decoder truth table

The type of instruction—Data-Processing Register, Data-Processing Immediate, STR, LDR, LDRB, or B—is determined by the Main Decoder. It provides the datapath with the proper control signals. ImmSrc_{1:0}, RegSrc_{1:0}, ALUSrc, and MemtoReg are sent straight to the datapath.

The LDRB implementation is displayed in the truth table above. The main change made to the decoder was to make room for LDRB instruction. We discovered that the primary decoder and ALU decoder needed to be changed, so we followed Dr. Wang's instructions and made the required changes to dmem.v and regfile.v. We started by changing the code in the decoder.v file using the table; the code for Code 1 is shown below. As demonstrated in Code 1, the case statement for the operation type Op for memory instruction now includes a new "if" statement that, if both the L and B bits are true, executes the LDRB instruction. The LDRB operation is avoided if L is 0. The LDR procedure is then carried out if B is 1. Lastly, the STR operation is carried out if both L and B are 0. The output control signal shows how the two instructions differ from one another. Since the ImmSrc for LDRB is 00, extend takes the first 8 bits (a byte) of the instruction's Src2 immediate field and adds 24 bits to it. Since ImmSrc for LDR is 01, extend

takes the 12-bit immediate and adds 20 bits to it.

```

always @(*) begin
  case(Op)
    // Data-processing immediate
    2'b00: if (Funct[5]) controls = 10'b00001010001;
    // Data-processing register
    else controls = 10'b00000010001;
    //LDRB
    2'b01: if (Funct[0] && Funct[2]) controls = 10'b00011111000;
    //LDR
    else if (Funct[0]) controls = 10'b00011110000;
    //STR
    else controls = 10'b10011100100;
    // B
    2'b10: controls = 10'b01101000010;
    // Unimplemented
    default: controls = 10'bx;
  endcase
end

```

Code 1. Main Decoder implementation

We implemented LDRB in the Main Decoder using the truth table and this generated control signal configures the processor to compute the effective memory address using the ALU, load a single byte from memory, and zero-extend it to 32 bits before writing it to the destination register. The added bit to RegW was added in the RegisterFile

```

always @(negedge clk) begin
  if (we3[1])
    case(we3[0])
      1'b0: rf[wa3] <= wd3;
      1'b1: rf[wa3] <= {24'b0, wd3[7:0]};
    endcase
end

```

Code 2. Additional byte used in register file implementation

The we3 signal is particularly useful for supporting instructions like LDR and LDRB. For LDR, we3 = 2'b01, writes the full 32-bit value from memory to the destination register. For LDRB, we3 = 2'b11, writes only LSB (least significant byte) from memory to the destination register, with the remaining bits zero-extend.

2. EOR

The behavior of the ALU decoder is given by the truth table below

ALUOp	Funct _{4:1} (cmd)	Funct ₀ (S)	Type	ALUControl _{2:0}	FlagW _{1:0}
0	X	X	Not DP	000 (ADD)	00
1	0100	0	ADD	000(ADD)	00
		1			11
	0010	0	SUB	001(SUB)	00
		1			11
	0000	0	AND	010(AND)	00
		1			10
	1100	0	ORR	011(ORR)	00
		1			10

	0001	0	EOR	100(EOR)	00
		1			10

Table 2. ALU Decoder truth table

The ALU Decoder selects ALUControl for data-processing instructions according to the kind of instruction. Additionally, it states that when the S-bit is set, FlagW will update the status flags.

```

always @(*) begin
case(ALUControl[2:0])
3'b000: Result = sum[31:0]; //ADD
3'b001: Result = sum[31:0]; //SUB
3'b010: Result = a & b; //AND
3'b011: Result = a | b; //ORR
3'b100: Result = a ^ b; //EOR
//default: Result = 32'b0;
endcase
end

```

Code 3. ALU Case statement for the ALU control signal implementation

The exclusive OR, or EOR, will be a new operation required by the ALU. This necessitates changing the 2-bit wide controller signal, ALUControl, to a 3-bit wide signal. The EOR command (Code 3, the additional code is boxed in red) can now be assigned utilizing the newly accessible signal lines. The revised ALUControl signal width must then be accommodated by modifying the datapath, decoder, controller, and ARM module. All you have to do is enter each file and change "[1:0] ALUControl" to "[2:0] ALUControl." To enable the use of EOR, the decoder ALUOp case statement was also modified (see Code 4). The CPU can now employ EOR.

```

// ALU Decoder
always @(*) begin
if (ALUOp) begin// determine which DP Instruction
case(Funct[4:1])
4'b0100: ALUControl = 3'b000; // ADD
4'b0010: ALUControl = 3'b001; // SUB
4'b0000: ALUControl = 3'b010; // AND
4'b1100: ALUControl = 3'b011; // ORR
4'b0001: ALUControl = 3'b100; //EOR
default: ALUControl = 3'bx; // unimplemented
endcase
end
end

```

Code 4. ALU Decoder implementation

Results and Evaluation

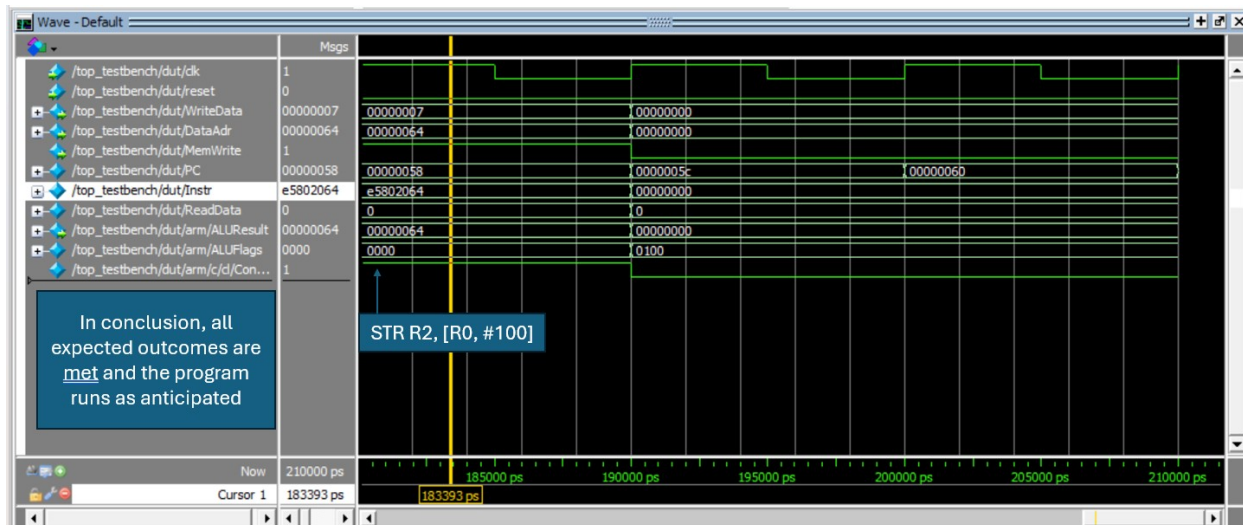
After completing the design of our single-cycle ARM processor and all the necessary implementations to the various Verilog files we created, our next step was to run the project using ModelSim. ModelSim creates waveforms for our project so we can visualize what exactly our program is doing at each step of the process. For task 2, our predictions for what the executing test program will look like is as follows:

Cycle	reset	PC	Label	Inst r	Src A	Src B	Branch	AluResult	Flags 3:0 [NZCV]	Cond Ex	Write Data	MemWrite	ReadData
1	1	0	MAIN	SUB R0, R15, R15 E04F00 0F	8	8	0	0	0100	1	8	0	X
2	0	4		ADD R2, R0, #5 E28020 05	0	5	0	5	0000	1	X	0	X
3	0	8		ADD R3, R0, #12 E28030 0C	0	C	0	C	0000	1	X	0	X
4	0	c		SUB R7, R3, #9 E24370 09	C	9	0	3	0000	1	X	0	X
5	0	10		ORR R4, R7, R2 E18740 02	3	5	0	7	0000	1	5	0	X
6	0	14		AND R5, R3, R4 E00350 4	C	7	0	4	0000	1	7	0	X
7	0	18		ADD R5, R5, R4 E08550 04	4	7	0	B	0000	1	7	0	X
8	0	1C		SUBS R8, R5, R7 E05580 07	B	3	0	8	0010	1	3	0	X
9	0	20		BEQ END 0A0000 0C	40	48	1	X	0000	1	0	0	X
10	0	24		SUBS R8, R3, R4 E05380 04	C	7	0	5	0010	1	7	0	X

11	0	28		BGE AROUND	48	0	1	X	0000	1	0	0	X
				AA0000 00									
12	0	2C		ADD R5, R0, #0	x	x	x	X	x	x	x	x	X
				E28050 00									
13	0	30	AROU ND	SUBS R8, R7, R2	3	5	0	D	0010	1	5	0	X
				E05780 02									
14	0	34		ADDLT R7, R5, #1	B	1	0	C	0000	1	X	0	X
				B28570 01									
15	0	38		SUB R7, R7, R2	C	5	0	7	0000	1	5	0	X
				E04770 02									
16	0	3C		STR R7, [R3, #84]	30	54	0	7	0000	1	7	1	X
				E58370 54									
17	0	40		LDR R2, [R0, #96]	34	60	0	7	0000	1	0	0	12
				E59020 60									
18	0	44		ADD R15, R15, R0	88	0	0	F	0000	1	0	0	X
				E08FF0 00									
19	0	48		ADD R2, R0, #14	x	x	X	x	x	x	x	x	X
				E28020 0E									
20	0	4C		B END	84	4	1	X	0000	1	X	0	X
				EA0000 01									
21	0	50		ADD R2, R0, #13	x	x	X	X	x	x	x	x	X
				E28020 0D									
22	0	54		ADD R2, R0, #10	X	x	x	x	x	x	x	x	X
				E28020 0A									
23	0		END	STR R2, [R0, #100]	0	64	0	64	0000	1	7	1	X

Table 3. Task 2- Test Program

Now after looking at the table, we will look at the waveform output from ModelSim that show the execution of the instructions ADD, SUB, AND, ORR, LDR, STR, B:



Waveform 1. Task 2- Last output; Rest shown in Appendix B

This is just one section of the waveform output, as the rest will be located in appendix B. This is the end result of the waveform output and as we can see, this matches what was expected from the table showing the executing test program. The team was able to get the program to write the correct output to address 100.

Further, we added the necessary implementations for the extra credit portion as well, and therefore we made an additional table representing our predictions once again of what will happen on each single. The extra credit instructions to be added were EOR and LDRB:

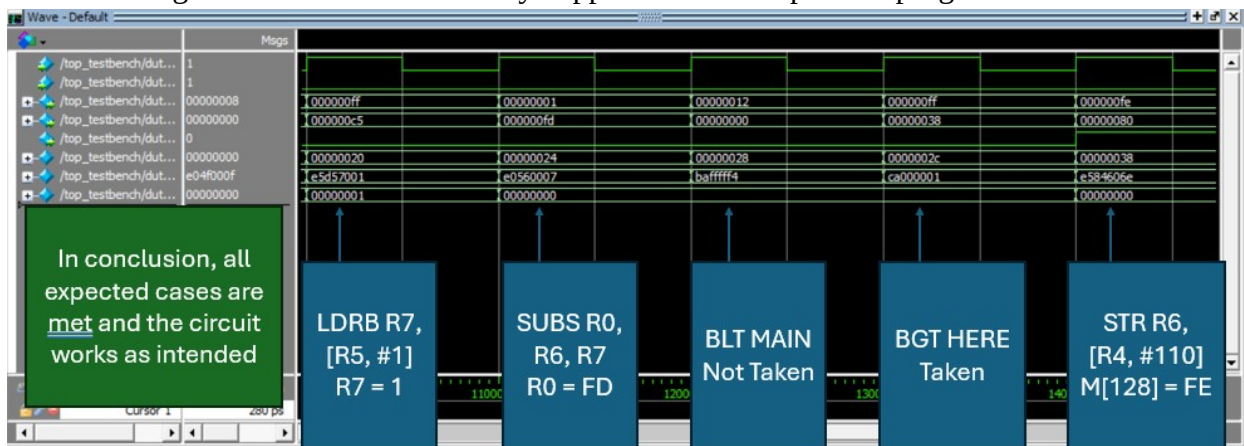
Cycle	reset	PC	Label	Inst r	Src A	Src B	Branch	Alu Result	Flags _{3:0} [NZCV]	CondEx	WriteData	Mem Write	ReadData
1	1	0	MAIN	SUB R0, R15, R15 E04F 000F	8	8	0	0	0100	1	8	0	X
2	0	4		ADD R1, R0, #255	0	FF	0	FF	0000	1	C	0	X

				E280 10FF									
3	0	8		ADD R2, R1, R1 E081 2001	F F	FF	0	1FE	0000	1	FF	0	X
4	0	C		STR R2, [R0, #196] E580 20C4	0	C4	0	C4	0000	1	0	1	X
5	0	1 0		EOR R3, R1, #77 E221 304D	F F	4D	0	B2	0000	1	0	0	X
6	0	1 4		AND R4, R3, #0x1 F E203 401F	B 2	1F	0	12	0000	1	1C	0	X
7	0	1 8		ADD R5, R3, R4 E083 5004	B 2	12	0	C4	0000	1	12	0	1FE
8	0	1 C		LDRB R6, [R5] E5D5 6000	C 4	0	0	C4	0000	1	0	0	1FE
9	0	2 0		LDRB R7, [R5, #1] E5D5 7001	C 4	1	0	C5	0000	1	FF	0	1
10	0	2 4		SUBS R0, R6, R7 E056 0007	F E	1	0	FD	0110	1	0	0	X
11	0	2 8		BLT MAIN BAFF FFF4	30	FFF FFFD 0	1	0	0110	0	X	0	X
12	0	2 C		BGT HERE CA00 0001	34	4	1	38	0000	1	X	0	X
13	0	3 0		STR R1, [R4, #110] E584 12E	X	X	x	x	x	x	x	x	x

14	0	3 4		B MAIN EAF FFF1	X	X	x	x	x	x	x	x	x
15	0	3 8	HE RE	STR R6, [R4, #110] E584 606E	12	6E	0	80	0000	1	0	1	x

Table 4. Extra Credit- Test Program

Once again, we go to ModelSim to view the output waveform of our program to get a better understanding and visualize what exactly happens in each step of the program.



Waveform 2. Extra Credit- Last output; Rest shown in Appendix C

Shown through our predictions table and the ModelSim output waveforms, our modified ARM processor did not break and works as anticipated. We believe we were able to get this program running as hoped, as all the tests that were carried out seemed to be successful.

Conclusion

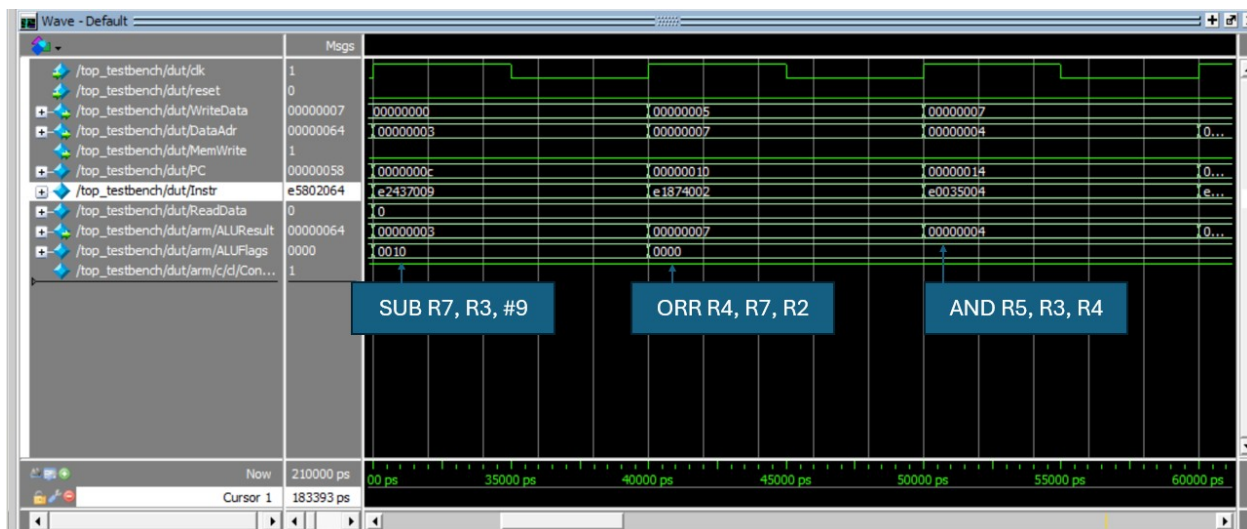
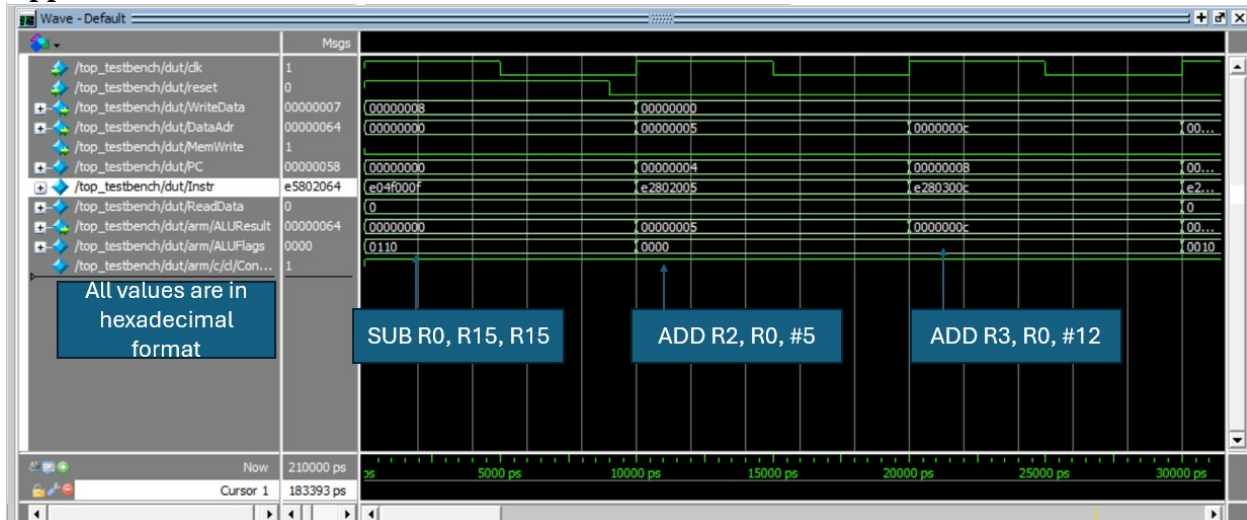
Throughout this process of building a single-cycle ARM processor, the team has been able to grasp a deeper understanding of all the different moving parts of what it takes to make the processor work. Our team was able to create a working single-cycle ARM processor with the help of our previous experiences working with various labs, including ones built for multiplexers, ALU logic and control, register files, instruction and data memory, datapath, and decoders. The team was able to take all of these individual works and combine them into one project to serve as our building blocks for what was to come next. We then were able to design and implement the various instructions and functions necessary to execute multiple instructions, including datapath instruction ADD, ORR, AND, and ORR, memory instructions LDR and STR, and branch instructions B. We then were able to run the test program in ModelSim to view our output waveforms and compare it to our prediction table to determine its' validity. As shown in previous sections, our predictions did indeed match our ModelSim waveform output after running it on the test program.

Furthermore, we were even able to take this project one step further by including two instructions, EOR and LDRB, in our processor design for extra credit. After modifying our single-cycle ARM processor design, we had to ensure we did not break the program or negatively affect the old instructions. We once again created a predictions table, so that we know what we are looking for in the waveform output. After running the program on ModelSim, we were able to view the waveform output and compare it to our expected results. We once again believe that our program ran correctly and worked as desired.

Garrett Good	Joyce Berdkan
• Headed the Report • Wrote abstract, introduction, background, Evaluation, Results, and Conclusion; helped in other sections as well • Helped with the code; wrote the comments on both task 2 and extra credit files • Wrote comments on waveform output for task 2 • Aided in creating the tables	• Headed the Coding in Quartus • Wrote Design, implementation, and specification sections; helped in other sections as well • Helped create the tables for task 2 and Extra credit • Wrote comments on waveform output for Extra credit

Overall, both members worked with writing the report, building the tables, and writing the necessary functions/files in Quartus. We met up throughout Thanksgiving break to ensure that we would both stay focused and on task with what the project required.

Appendix A



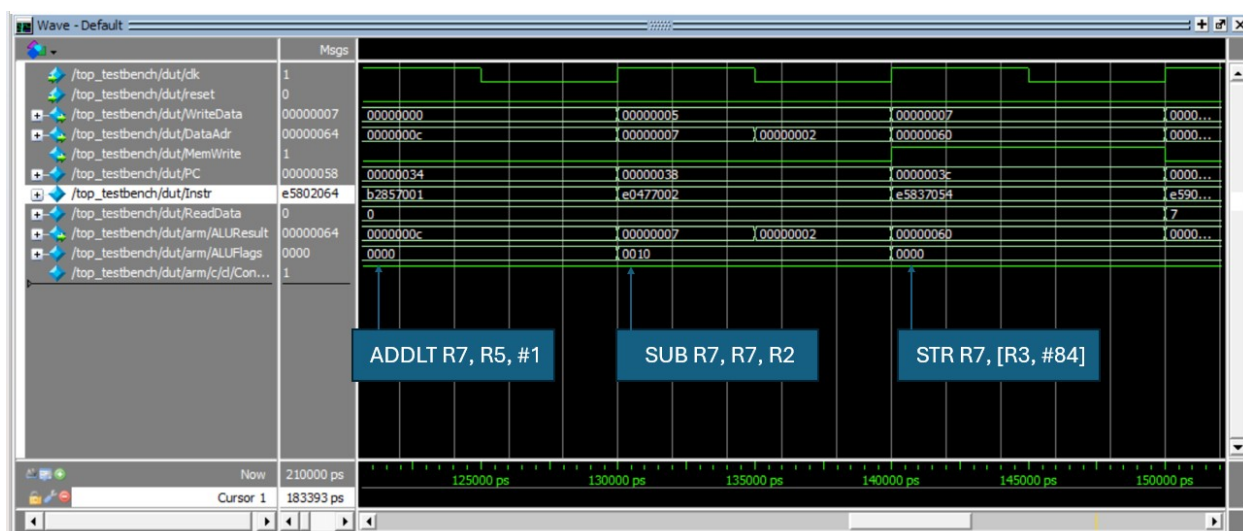
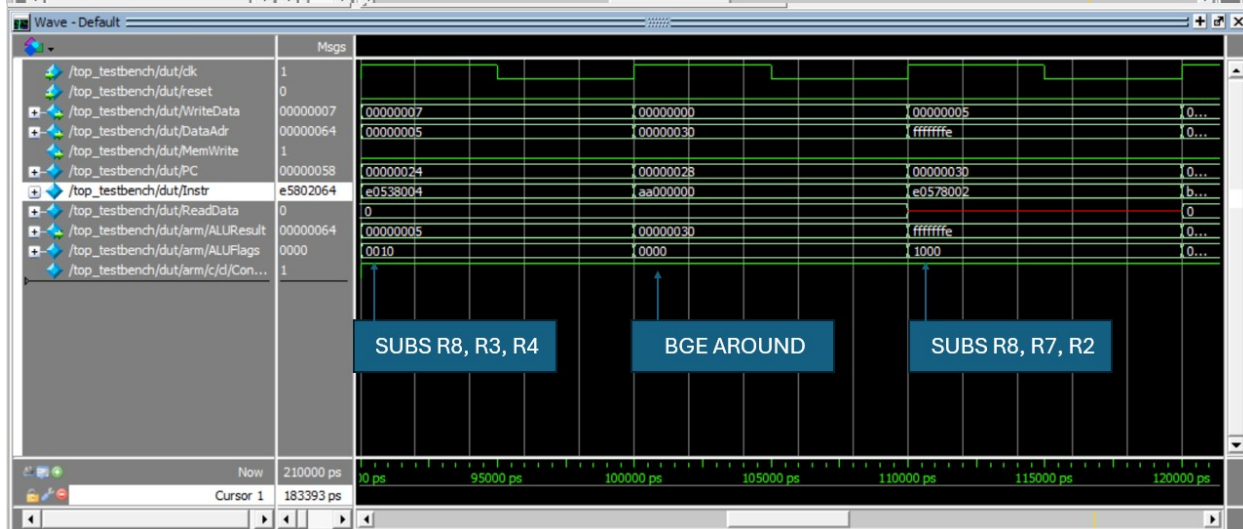
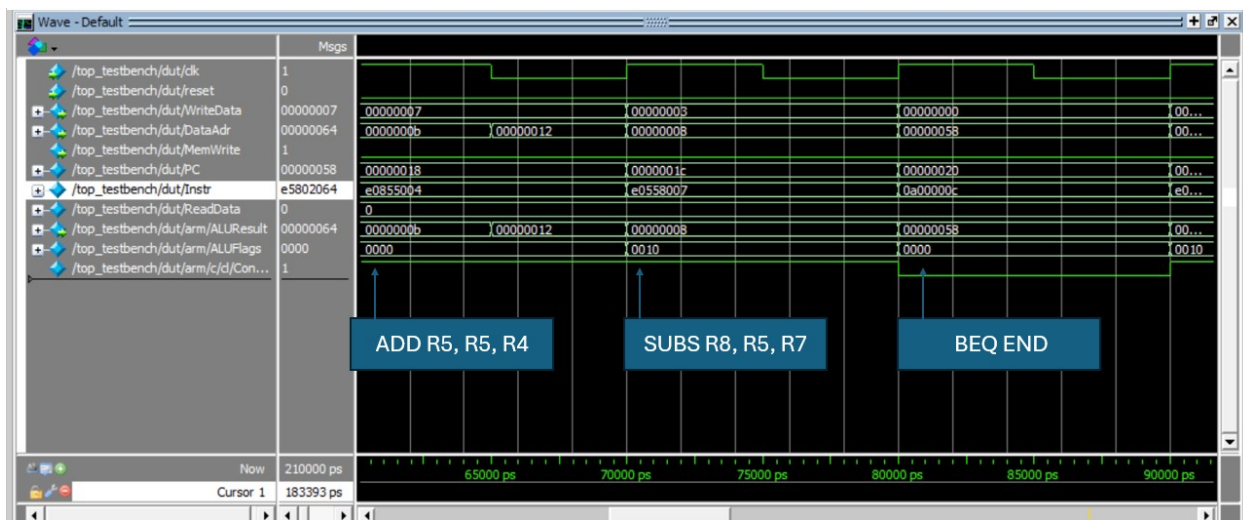




Figure A-1. Waveforms from ModelSim for Task 2

Appendix B

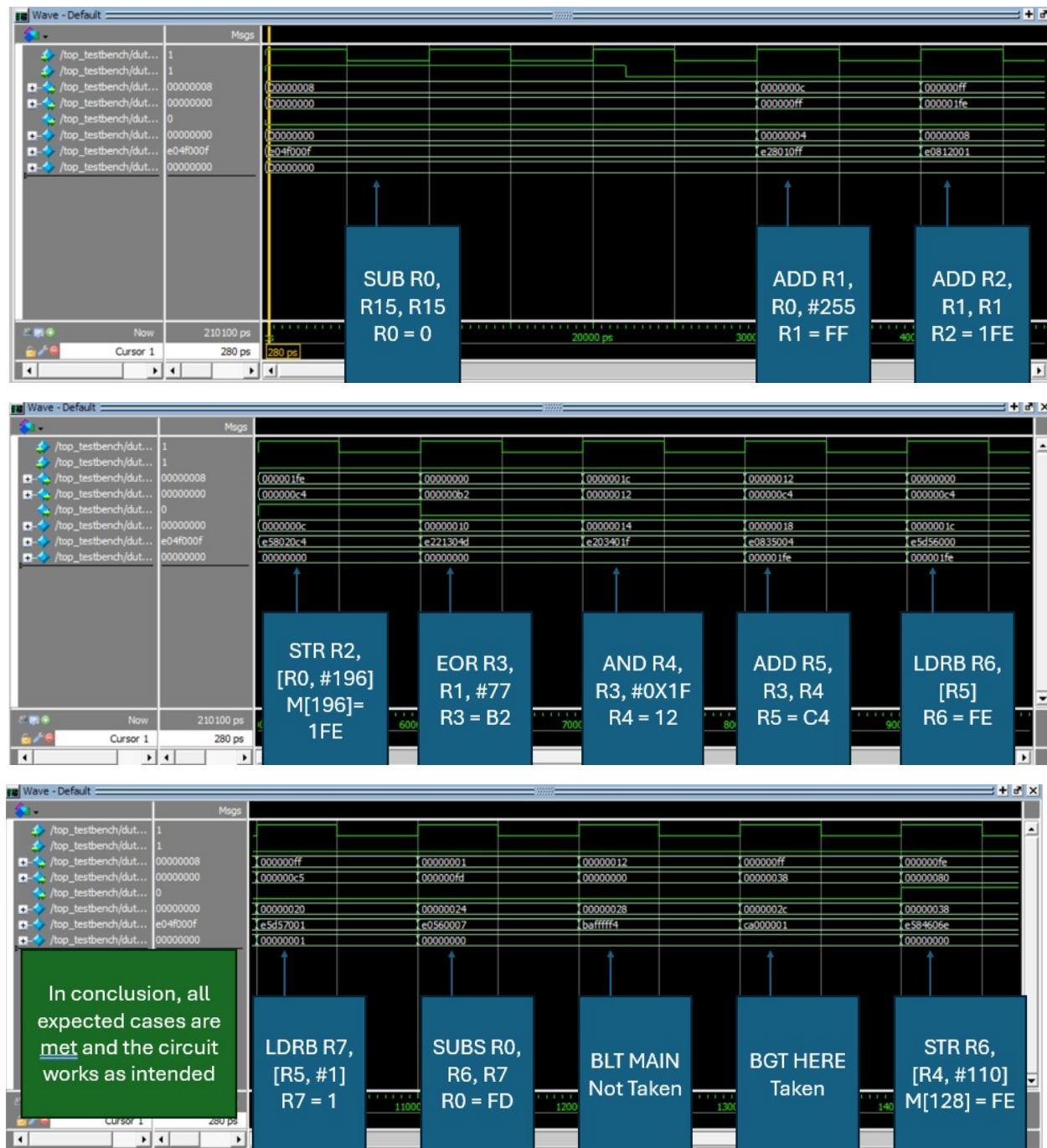


Figure B-1. Waveforms from ModelSim for Extra Credit